

UE2 Theorie-Fragen & Antworten — Exercise 2

Laufende Sammlung, gleiches Schema wie [../ue1/UE1_THEORIE_ANTWORTEN.md](#).

Demo 1 — Package Manager & Projekt-Metadaten

Gewählt: **npm** (eigene Vorerfahrung). `package.json` sauber ausgefüllt (siehe `UE2_CHANGES.md` für die Feld-für-Feld-Begründung), `.gitignore` um `node_modules/` / `dist/` / `.env*` erweitert, `dayjs` als erste echte Abhängigkeit installiert.

F1: Welches Problem löst ein Package Manager wirklich, das „Bibliothek runterladen und in einen Ordner legen“ nicht löst? Konkret werden.

Einfach gesagt: Beim reinen Runterladen musst du selbst Buch führen — welche Version hab ich, braucht das noch was anderes, gibt's was Neueres. npm macht das automatisch, wie ein Lagerverwalter statt eigener Zettelwirtschaft.

Ganz konkret an `dayjs`:

- **Reproduzierbarkeit über Versionen.** `package.json` sagt „irgendeine 1.x-Version ab 1.11.23“ (`^1.11.23`), die Lockfile friert die *exakte* installierte Version ein. `npm install` liefert bei mir, bei dir, und in der CI **dieselbe** Version. Ein manuell heruntergeladenes `dayjs.min.js` sagt niemandem, welche Version das war oder ob es die neueste/eine gepatchte ist.
- **Transitive Abhängigkeiten.** `dayjs` hat zufällig keine eigenen Abhängigkeiten — die meisten echten Pakete aber schon. Lädt man „nur die eine Datei“ von Hand, bekommt man *nicht* automatisch deren eigene Abhängigkeiten mit, geschweige denn in kompatiblen Versionen zueinander. npm löst diesen ganzen Baum automatisch auf.
- **Updates & Sicherheit.** `npm update` / `npm audit` sagen mir, ob eine installierte Version eine bekannte Sicherheitslücke hat und was ich upgraden müsste. Bei handkopierten Dateien weiß niemand, ob irgendwo im Projekt eine drei Jahre alte, verwundbare Kopie liegt.
- **Sauberes Entfernen.** `npm uninstall dayjs` entfernt exakt das, was `npm install dayjs` hinzugefügt hat (laut Lockfile). Von Hand kopierte Dateien hinterlassen nie ganz klar, was zu welcher „Installation“ gehörte.
- **Isolation pro Projekt.** `node_modules/` ist lokal für *dieses* Projekt. Ein anderes Projekt auf demselben Rechner kann eine andere, sogar inkompatible Version derselben Bibliothek haben, ohne dass sich beide in die Quere kommen — bei einer globalen „Bibliotheks-Ordner“-Lösung wäre das nicht so einfach.

F2: Unterschied `dependencies` vs. `devDependencies`? In welche Kategorie kommen Vite, Linter/Formatter und TypeScript — und warum?

Einfach gesagt: Werkzeugkiste (Hammer, Bohrmaschine — bleibt in der Werkstatt) vs. das fertige Haus selbst (Ziegel, Fenster — steckt drin). `devDependencies` = Werkzeug zum Bauen, `dependencies` = das, was im fertigen, ausgelieferten Produkt tatsächlich mitläuft.

- `dependencies`: Code, den die **ausgelieferte, laufende App** zur Laufzeit braucht — bei einer Browser-App: alles, was am Ende tatsächlich im Bundle landet, das im Browser der Nutzer:innen ausgeführt wird. `dayjs` gehört hierher, *sobald* wir es tatsächlich in App-Code importieren, der im Browser läuft.
- `devDependencies`: Werkzeuge, die nur **während der Entwicklung/des Builds** gebraucht werden — laufen auf meinem Rechner bzw. in der CI, aber ihr eigener Code landet nie im Browser der Nutzer:innen. **Vite** (baut/serviert die App, läuft selbst nie im Browser), **ESLint/Prettier** (prüfen/formatieren Quelltext, laufen nie im Browser), **TypeScript** (Compiler, übersetzt zu JS, das Ergebnis läuft im Browser — der Compiler selbst nie) gehören alle hierher.

Der praktische Unterschied: `npm install --omit=dev` (z. B. auf einem reinen Deploy-Zielsystem) installiert nur `dependencies` — korrekt, denn ein Deployment-Ziel braucht keinen Linter. Test dafür, in welche Kategorie etwas gehört: „läuft der Code davon im Browser der Endnutzer:innen mit?“ Ja → `dependencies`. Nein, nur beim Bauen/Entwickeln → `devDependencies`.

F3: Wofür ist eine Lockfile da, und was könnte für Teammitglieder (oder die CI) schiefgehen, wenn sie nicht committet wäre?

Einfach gesagt: `package.json` ist ein Rezept, das nur "füg Mehl hinzu" sagt (ungenau). Die Lockfile ist dasselbe Rezept mit exakten Grammangaben. Nur mit der Lockfile bekommen alle garantiert denselben "Kuchen".

`package.json` gibt **Bereiche** an (`^1.11.23` erlaubt jede spätere `1.x.x`-Version). Die Lockfile (`package-lock.json`) friert den **exakt aufgelösten Baum** ein — jede einzelne (auch transitive) Version plus einen Integrity-Hash, der die heruntergeladene Datei gegen Manipulation prüft. `npm install` mit vorhandener Lockfile reproduziert exakt diesen Baum; ohne Lockfile löst npm die Bereiche bei **jedem** `install` neu auf.

Ohne committete Lockfile, konkret an `dayjs`: veröffentlicht dayjs morgen `1.12.0` (erlaubt durch `^1.11.23`), bekommt ein Teammitglied oder die CI beim Klonen + `npm install` eine **andere** Version als die, mit der der Code geschrieben und getestet wurde. Folgen: „geht bei mir, geht bei dir nicht“-Bugs, eine ungetestete transitive Versions-Änderung bricht etwas oder bringt eine Schwachstelle mit, ein Build ist lokal grün und in der CI rot (oder umgekehrt) — rein durch Versions-Drift, nicht durch einen echten Code-Fehler. Und: die Integrity-Hashes (Schutz gegen ein manipuliertes/kompromittiertes Registry-Paket) gehen mit verloren.

F4: npm gewählt — was würde man bei einem größeren Projekt durch pnpm gewinnen/verlieren?

Einfach gesagt: pnpm hebt jeden Baustein nur **einmal** auf der ganzen Festplatte auf (statt einmal pro Projekt) und leiht ihn sich aus, wenn ein Projekt ihn braucht — spart Platz und Zeit. Kostet: ein Zusatzprogramm, das npm (immer mit Node dabei) nicht braucht.

Gewinn durch pnpm:

- **Ein globaler, inhaltsadressierter Store** auf der Platte: eine physische Kopie von z. B. `dayjs@1.11.23` liegt **einmal** auf dem Rechner und wird über Hardlinks in jedes Projekt „eingehängt“, das genau diese Version braucht — statt (wie bei npm klassisch) in jedem Projekt eine eigene volle Kopie von `node_modules` zu haben. Bei vielen Node-Projekten auf einer Maschine: massiv weniger Plattenplatz, spürbar schnellere Installs.
- **Striktes `node_modules`-Layout:** pnpm verlinkt so, dass ein Paket nur das sehen kann, was es *selbst* explizit als Abhängigkeit deklariert hat. Das deckt sogenannte „Phantom Dependencies“ auf — Code, der nur zufällig funktioniert, weil irgendein *anderes* Paket dieselbe Bibliothek zufällig mitbringt, obwohl man sie selbst nie in `package.json` eingetragen hat. Bei npm's klassisch „flachem“ `node_modules` ist das unsichtbar und bricht überraschend, sobald sich die andere Abhängigkeit ändert — bei größeren Teams ein reales, wiederkehrendes Problem.

Verlust/Kosten:

- pnpm ist ein **zusätzliches Werkzeug**, das separat installiert und in der CI genauso verfügbar gemacht werden muss — npm ist dagegen mit jeder Node-Installation garantiert vorhanden (genau das Argument aus Task 1 für npm).
- Vereinzelt (selten, aber real) Kompatibilitätsprobleme mit älteren Tools, die stillschweigend ein *flaches* `node_modules` voraussetzen.
- Migration später ist mechanisch möglich (`package.json`-Semantik bleibt gleich, nur `package-lock.json` → `pnpm-lock.yaml`), aber jedes Teammitglied + die CI-Konfiguration müssen mitgezogen werden.

Demo 2 — Vite als Dev-Server

`public/` -Umzug (`data/`, `assets/`), `vite.config.js`, `"dev": "vite"`, Dev-Server auf allen 5 Views geprüft, HMR mit CSS- und JS-Änderung live gegenübergestellt. Details in `UE2_CHANGES.md`.

F1: Unterschied zwischen dem alten statischen Server und Vites Dev-Server? Mindestens eine Sache nennen, die Vite tut und ein reiner statischer Server nicht.

Einfach gesagt: der alte Server ist ein stummer Briefträger — liefert nur Dateien aus, wie sie daliegen. Vite ist ein schlauer Assistent — merkt Änderungen automatisch, kennt installierte Pakete, kompiliert nur bei Bedarf.

Der alte Server (`python -m http.server` aus UE1) macht **genau eine Sache**: eine angefragte Datei byte-für-byte so ausliefern, wie sie auf der Platte liegt. Er weiß nichts von `import`-Anweisungen, npm-Paketen oder Modul-Abhängigkeiten.

Vites Dev-Server tut mehrere Dinge, die ein reiner statischer Server nicht kann — im Netzwerk-Log konkret beobachtet:

- Er liefert `/@vite/client` aus — eine Datei, die in unserem Quellcode **gar nicht existiert**. Vite injiziert sie beim Ausliefern von `index.html` on-the-fly. Dieses Skript baut die WebSocket-Verbindung für HMR auf.
- Er liefert `/node_modules/vite/dist/client/env.mjs` aus — direkt aus `node_modules`, ohne dass wir das irgendwo referenziert haben. Ein reiner Dateiserver hat kein Konzept von "installierten Paketen", die er bei Bedarf auflösen und ausliefern kann.
- Er versteht **bare imports** wie `import dayjs from "dayjs"` (kommt in Demo 6 dran) — löst den Paketnamen zu einer echten Datei in `node_modules` auf. Ein statischer Server würde bei `GET /dayjs` schlicht mit `404` antworten, weil es lokal keine Datei mit diesem Namen gibt.
- Er kompiliert/transformiert **on demand**, nur die Datei, die gerade angefragt wird — kein Vorab-Bundle des ganzen Projekts, bevor der Server überhaupt startet.

F2: Was ist Hot Module Replacement, und was genau hast du beobachtet (und was nicht, z. B. beim App-Zustand)?

Einfach gesagt: HMR = **H**ot **M**odule **R**eplacement, "heißer Teil-Austausch" — während die Seite noch läuft (heiß), wird ein einzelner Baustein (Modul) ausgetauscht, ohne alles andere anzufassen. Wie wenn jemand dir beim Lesen nur eine korrigierte Buchseite reinschiebt, statt dir das ganze Buch wegzunehmen und du bei Seite 1 neu anfangen musst (= kompletter Reload).

HMR heißt: wenn sich eine Datei ändert, schickt der Dev-Server über die schon offene WebSocket-Verbindung **nur das geänderte Modul** (bzw. bei CSS: das geänderte Stylesheet) an die bereits laufende Seite, und der im Browser laufende Vite-Client tauscht es **an Ort und Stelle** aus — ohne das Dokument neu zu laden. Der komplette JS-Ausführungskontext (Variablen, In-Memory-Zustand, aktuelle Ansicht) bleibt dabei erhalten.

Beobachtet (CSS-Änderung, `--color-accent` in `styles.css`):

- Konsole: `[vite] css hot updated: /styles.css`.
- Netzwerk: genau ein neuer Request, `GET /styles.css?t=<timestamp>` (Cache-Busting-Query) — kein Navigations-Request.
- Die neue Farbe war sofort sichtbar.
- **Was nicht passiert ist:** eine vorher gesetzte globale Variable (`window.__hmrMarker`) war danach immer noch da, `state.bookmarks` (In-Memory-App-Zustand) unverändert, aktueller Hash (`#evidence`) unverändert. Kein Reload — der JS-Kontext lief einfach weiter.

Beobachtet (JS-Modul-Änderung, Kommentar in `js/views/dashboard.js`):

- Konsole: **erneut** `[vite] connecting... / connected`. — der Client baut seine WebSocket-Verbindung neu auf. Das passiert nur bei einem echten Full-Reload.
- **Was diesmal verloren ging:** die vorher gesetzte Marker-Variable war weg (`undefined`), ein zuvor geöffnetes Beweisstück-Detail (`state.selectedEvidence`, nie in `localStorage` gespeichert) war nach dem Reload wieder geschlossen.

Warum der Unterschied: unsere Module haben keinerlei `import.meta.hot.accept()`-Code. CSS-HMR bekommt man bei Vite automatisch ohne eigenes Zutun; JS-HMR mit Zustandserhalt braucht dagegen explizites Opt-in im Modul selbst (oder ein Framework, das das für einen erledigt — z. B. React/Vue über ihre Vite-Plugins). Ohne das fällt Vite bewusst auf einen sauberen Full-Reload zurück, statt ein Modul zu ersetzen und dabei möglicherweise einen inkonsistenten Zustand zu riskieren.

F3: Warum integriert sich eine schon in ES-Module gesplittete App natürlich mit einem Tool wie Vite, verglichen mit der ursprünglichen Ein-`<script>`-Version?

Einfach gesagt: Vite muss genau wissen, welcher einzelne Baustein sich geändert hat, um nur den auszutauschen. Das kann es nur, weil unsere App seit UE1 in benannte, über

`import / export`

verbundene Bausteine aufgeteilt ist. Bei der alten, einen riesigen `app.js`-Datei hätte Vite nicht erkennen können, "welcher Teil" sich geändert hat — es hätte immer alles neu laden müssen.

Vites komplettes Dev-Serving- und Bundling-Modell **basiert** auf dem nativen ES-Modul-Graphen: `<script type="module" src="...">` plus `import / export`-Anweisungen geben Vite einen echten, statisch analysierbaren Abhängigkeitsbaum. Dadurch kann Vite:

- jede Datei **einzel**n, **on demand** ausliefern (im Netzwerk-Log genau so beobachtet — 13 einzelne `js/*.js`-Requests statt einer riesigen Datei),
- bei einer Änderung **exakt wissen, welches eine Modul betroffen ist** und nur das neu schicken (die Grundlage für HMR überhaupt),
- beim Produktions-Build (Demo 3) **Tree-Shaking/Code-Splitting** machen, weil die Abhängigkeiten explizite, statische Deklarationen sind statt impliziter globaler Reihenfolge.

Die alte `app.js` (klassisches `<script src="app.js">`, eine 1000+-Zeilen-Datei ohne `import / export`) hätte Vite dagegen nur als **eine beliebige statische Datei** gesehen — kein Abhängigkeitsgraph, keine Modul-Grenzen, keine Möglichkeit herauszufinden, "welcher Teil hat sich geändert". Jede Änderung hätte einen kompletten Reload gebraucht, genau wie bei einem reinen Dateiserver — Vites eigentliche Stärken (granulares HMR, Dependency-Pre-Bundling, späteres Tree-Shaking) hätten schlicht nichts, woran sie andocken könnten. Der ES-Modul-Split aus UE1 ist also nicht nur "sauberer Code", sondern die **Voraussetzung**, die diese Übung überhaupt erst sinnvoll macht — genau das sagt auch die Angabe eingangs ("you need the ES-module split from that exercise finished first").

Demo 3 — Produktions-Build & Preview

`npm run build` (→ `vite build`) untersucht, `dist/` inspiziert, mit `npm run preview` End-to-End getestet, Quelle vs. Build für `js/`-Bundle und `index.html` verglichen. Details in `UE2_CHANGES.md`.

F1: Nenne mindestens drei konkrete Transformationen, die Vite beim Produktions-Build auf deinen Quellcode angewendet hat (Bundling, Minifizierung, gehashte Dateinamen — die nennen, die du wirklich beobachtet hast).

Einfach gesagt: aus 13 lesbaren Dateien wurde 1 kleine, unleserliche Datei mit komischem Namen. Drei Dinge sind dabei passiert: zusammengeklebt, zusammengequetscht, umbenannt.

Konkret bei uns beobachtet:

1. **Bundling.** Unsere 13 einzelnen `js/*.js`-Dateien (51.197 Bytes zusammen) wurden zu **einer** Datei `dist/assets/index-B1hg0Ru0.js` zusammengefasst. Statt 13 Netzwerk-Anfragen im

- Dev-Modus gibt's im Build nur noch 1.
2. **Minifizierung.** Diese eine Datei ist nur noch 23.269 Bytes groß (**-54 %**) und steht komplett auf **einer einzigen Zeile** (0 Zeilenumbrüche, gegenüber z. B. 111 lesbaren Zeilen in `js/data.js`). Kommentare weg, Variablennamen wie `state / caseRes` zu `e / t / n` eingedampft.
 3. **Content-Hashing der Dateinamen.** `js/main.js` (fester Name) wird zu `index-B1hg0Ru0.js` (Name hängt vom exakten Inhalt ab) — Details bei F2.
 4. (Bonus, auch beobachtet) **Umschreiben der Referenzen in `index.html`.** `<script type="module" src="js/main.js">` (stand am Ende von `<body>`) wurde nach `<head>` verschoben und zu `<script type="module" crossorigin src="/assets/index-B1hg0Ru0.js">` umgeschrieben — analog für das Stylesheet. **Nicht** umgeschrieben wurde dagegen ``, weil das eine `public/`-Datei ist (siehe Demo 2) — Vite lässt `public/`-Referenzen bewusst unangetastet.

F2: Warum enthalten Produktions-Dateinamen typischerweise einen Content-Hash? Welches Problem löst das bei echten Deployments?

Einfach gesagt: wie ein Chargen-Code auf einer Lebensmittelpackung — ändert sich das Rezept, ändert sich die Nummer. So verwechselt niemand (auch nicht der Browser) alte und neue Version.

Browser (und dazwischengeschaltete CDNs) wollen Dateien **möglichst lange zwischenspeichern** (cachen), um bei wiederkehrenden Besucher:innen nicht ständig alles neu herunterladen zu müssen

— das spart massiv Ladezeit. Das Problem dabei: **woher weiß der Browser, wann sich eine Datei**

tatsächlich geändert hat, ohne bei jedem Besuch nachzufragen (was den Cache-Vorteil wieder zunichtemachen würde)?

Die Lösung: der Dateiname selbst wird aus dem **exakten Inhalt** der Datei berechnet (`index-B1hg0Ru0.js`). Ändert sich am Code auch nur ein Zeichen, ändert sich der ganze Hash, also

der ganze Dateiname. Das erlaubt zwei Dinge gleichzeitig, die sich sonst widersprechen würden:

- Man kann dem Browser sagen "cache `index-B1hg0Ru0.js` für immer, frag nie wieder nach" (extrem lange Cache-Zeiten, z. B. ein Jahr) — **ohne** Risiko, weil dieser exakte Name garantiert nie einen anderen Inhalt bekommt.
- Ändert sich der Code, bekommt die neue Version automatisch einen **neuen** Namen — der Browser hat den alten Namen zwar noch gecacht, aber `index.html` verweist jetzt auf den neuen Namen, also lädt er zwangsläufig frisch nach.

Ohne Hash müsste man entweder sehr kurze Cache-Zeiten setzen (langsamer für alle) oder riskieren,

dass jemand tagelang eine veraltete, kaputte Version der Seite aus dem eigenen Browser-Cache ausgeliefert bekommt, obwohl längst ein Fix online ist.

F3: Warum würde man den Dev-Server selbst (`vite dev/vite`) nie für echte Nutzer:innen deployen, auch wenn er "funktioniert"?

Einfach gesagt: eine Baustelle "funktioniert" auch — man kann durchlaufen und alles sehen. Trotzdem lädt man da keine Gäste ein. Der Dev-Server ist für *dich beim Programmieren* gebaut, nicht für fremde Besucher:innen.

Konkret, direkt aus dem Vergleich unserer beiden Netzwerk-Logs:

- **Viel mehr Daten, viel mehr Requests.** Dev-Server: 13 einzelne, unminifizierte `.js`-Dateien (51 KB gesamt) plus `@vite/client` plus ein `node_modules`-Request. Preview/Produktion: 1 Datei (23 KB) plus 1 CSS-Datei. Für echte Besucher:innen (oft mit langsamerem Internet, auf dem Handy) ist das ein spürbarer Geschwindigkeitsunterschied.
- **Unnötige Entwickler-Maschinerie.** Der Dev-Server baut aktiv eine WebSocket-Verbindung für HMR auf (`[vite] connecting...`) — komplett nutzlos für jemanden, der nur deine Webseite lesen will, aber zusätzlicher Code, zusätzliche offene Verbindung, unnötige Angriffsfläche.
- **Nicht für echten Betrieb gebaut/gehärtet.** Der Dev-Server ist für den Iterations-Loop einer einzelnen Person beim Programmieren optimiert (schneller Start, hilfreiche, ausführliche Fehlermeldungen) — nicht dafür, viele gleichzeitige, fremde Besucher:innen zuverlässig und sicher zu bedienen.
- **Mehr Einblick als nötig.** Im Dev-Modus liegt der Code offen und lesbar da (mit Dateinamen, Kommentaren, Struktur) — im Produktions-Build ist er gebündelt/minifiziert. Für fremde Besucher:innen sollen nur so viele Interna wie nötig sichtbar sein.

Live beobachtet: über `vite preview` war die Konsole **komplett leer** (kein `[vite] connecting...` mehr) — der Beweis, dass die ganze Dev-Maschinerie im echten Build gar nicht mehr da ist.

Demo 4 — Lint & Format

ESLint + Prettier installiert und konfiguriert, `lint/lint:fix/format`-Scripts ergänzt, eine echte ungenutzte Variable gefunden (und für die Präsentation als Ein-Klick-Demo geparkt), Prettier auf den kompletten, bisher nie formatierten Code losgelassen (13 Dateien geändert, rein kosmetisch). Details in `UE2_CHANGES.md`.

F1: Unterschied zwischen dem, was ein Linter prüft/fixt, und dem, was ein Formatter prüft/fixt? Je einen konkreten Fund aus diesem Projekt nennen.

Einfach gesagt: Lektor (Linter, prüft Inhalt/Logik) vs. Schriftsetzer (Formatter, prüft nur die Optik). Beide verbessern den Text, aus komplett unterschiedlichen Gründen.

- **Linter (ESLint) prüft Logik/Verhalten** — Dinge, die vermutlich echte Fehler sind, unabhängig davon, wie der Code aussieht. **Konkreter Fund hier:** `js/views/dashboard.js`, eine absichtlich eingebaute ungenutzte Variable → `'unreviewedCount' is assigned a value but never used`

`no-unused-vars`. Das hat nichts mit Formatierung zu tun — die Zeile hätte perfekt eingerückt und mit den richtigen Anführungszeichen dastehen können, der Linter hätte trotzdem gemeckert.

- **Formatter (Prettier) prüft/erzwingt nur das Aussehen** — Einrückung, Zeilenumbrüche, Anführungszeichen, Kommas. **Konkreter Fund hier:** `js/utls.js` hatte eine Zeile, die länger als unser eingestelltes `printwidth: 100` war — Prettier hat sie automatisch umgebrochen und in Klammern gesetzt, ohne dass sich am Ergebnis (dem zurückgegebenen String) auch nur ein Zeichen geändert hat.

Die beiden überschneiden sich bei uns bewusst **nicht**: `eslint-config-prettier` schaltet die paar ESLint-eigenen Optik-Regeln ab, damit die zwei Werkzeuge sich nicht gegenseitig widersprechende Anweisungen geben.

F2: Warum sind `lint` und `lint:fix` zwei getrennte Scripts statt einem, das immer automatisch fixt? Wann willst du bewusst die nicht-fixende Version?

Einfach gesagt: nicht jeder gefundene Fehler ist gefahrlos automatisch reparierbar. Eine ungenutzte Variable könnte ein Tippfehler sein (löschen ist sicher) — oder ein Zeichen, dass du vergessen hast, sie irgendwo zu **benutzen** (löschen würde dann einen echten Bug verstecken). Das kann nur ein Mensch entscheiden.

Genau das haben wir live gesehen: `npm run lint:fix` auf unseren absichtlichen Fehler angewendet

— die Meldung blieb **trotzdem stehen**, `no-unused-vars` gehört zu den Regeln, die ESLint bewusst **nicht** automatisch anfasst, weil eine automatische Lösung (Variable löschen ODER an irgendeiner Stelle plötzlich "benutzen") in beide Richtungen falsch sein könnte.

Wann willst du bewusst die nicht-fixende Version (`lint`) statt `lint:fix`?

- **In der CI** (Demo 8): ein automatischer Prüflauf soll **nie heimlich deinen Code umschreiben** und das Ergebnis committen — er soll nur **melden**, dass etwas nicht stimmt, und den Build rot färben. Ein Mensch entscheidet dann, wie der Fix aussieht.
- **Beim Reviewen fremden Codes**, wenn du erstmal nur sehen willst, *was* gemeldet wird, bevor du irgendetwas automatisch veränderst.
- **Immer dann**, wenn ein gemeldetes Problem (wie eine ungenutzte Variable) mehrdeutig ist und eine automatische Lösung genauso gut die falsche sein könnte wie die richtige.

F3: Was macht `npm run lint` (bzw. `pnpm lint`) eigentlich "unter der Haube"? Wo sucht npm nach dem `lint`-Kommando, und würde es funktionieren, wenn dein Linter nicht als Projekt-Abhängigkeit, sondern nur global installiert wäre?

Einfach gesagt: npm schaut in `package.json` unter `scripts.lint` nach, findet den Text `"eslint ."`, und führt den aus — dabei schaut es **zuerst** im Projekt selbst nach einem passenden Programm, nicht irgendwo auf dem ganzen Rechner.

Genauer: `npm run lint` liest den String bei `scripts.lint` in `package.json` (`"eslint ."`) und führt ihn wie einen Terminal-Befehl aus. Damit das mit dem *unqualifizierten* Namen `eslint` funktioniert (kein Pfad davor), hängt npm für die Dauer dieses einen Befehls den Ordner `node_modules/.bin/` **vorne** an den Suchpfad (`PATH`) an. Genau in diesem Ordner legt npm beim Installieren jedes Pakets, das ein Kommandozeilen-Werkzeug mitbringt (wie `eslint` oder `vite`), automatisch eine kleine ausführbare Verknüpfung an. Deshalb findet `npm run lint` unser lokal installiertes ESLint, ganz ohne dass wir irgendwo einen Pfad dazu angeben mussten.

Würde es auch mit einer nur global installierten Version funktionieren? Technisch oft ja — wenn im Projekt selbst kein `node_modules/.bin/eslint` existiert, fällt die normale Shell-Pfadsuche auf den Rest deines System-`PATH` zurück, und ein global installiertes `eslint` würde dort gefunden. **Aber das wäre die falsche, fragile Lösung:**

- Ein Teammitglied oder die CI, das/die dieses globale Programm nicht installiert hat, bekommt schlicht `eslint: command not found` — das Projekt ist auf einmal nicht mehr „von selbst“ lauffähig, nur noch mit stillem Vorwissen über die eigene Maschine.
- Selbst wenn alle zufällig irgendein globales ESLint haben, könnten das **unterschiedliche Versionen** sein — dieselbe Versions-Drift-Problematik wie bei einer fehlenden Lockfile in Demo 1 ("bei mir meldet der Linter das, bei dir nicht").

Deshalb installieren wir ESLint/Prettier als `devDependencies`: sie werden dadurch Teil des reproduzierbaren Projekt-Setups (in `package.json` + `package-lock.json` festgehalten, genau wie `dayjs` oder `vite`), nicht eine zufällige Eigenschaft von irgendjemandes Rechner.

Demo 5 — TypeScript-Einstieg & erste Konvertierungen

TypeScript installiert, `tsconfig.json` mit bewusst gewählten Einstellungen, `js/utls.ts` + `js/lookup.ts` konvertiert (0 Fehler, kein `any`), `tsc --noEmit` in `npm run build` eingehängt. Unterwegs 7 echte, lehrreiche Typfehler gefunden und sauber gelöst (nicht mit `any` übertüncht). Details in `UE2_CHANGES.md`.

F1: Was schaltet `strict` in `tsconfig.json` eigentlich ein? Mindestens zwei einzelne Prüfungen nennen, sagen ob du es an gelassen hast und warum.

Einfach gesagt: `strict` ist ein Sammelschalter für ca. acht einzelne strengere Prüfungen auf einmal. Wir haben ihn an gelassen, weil genau zwei davon in unserem Code sofort real etwas gebracht haben.

`strict: true` schaltet unter anderem ein:

- `strictNullChecks` — `null` / `undefined` werden nicht mehr stillschweigend als Teil von *jedem* Typ akzeptiert; ein Wert, der fehlen kann, muss das explizit im Typ tragen (`string | undefined`), und jede Stelle, die ihn benutzt, muss das vorher abfangen.
Direkt bei uns erlebt: `evidenceMentionsPerson` hat `personIds?: string[]` (optional) im Interface. Ohne `strictNullChecks` wäre `ev.personIds.indexOf(...)` einfach so durchgegangen;
mit hätte TypeScript diese Zeile abgelehnt, wenn der vorhandene `if (!ev.personIds)` `return false;`-Guard nicht schon dagestanden hätte. Der Guard war schon vorher aus JS-Vorsicht da — mit `strictNullChecks` wird er vom "guten Stil" zur **Pflicht**.
- `noImplicitAny` — jeder Wert ohne erkennbaren Typ (z. B. ein Funktionsparameter ohne Annotation) wird zum Fehler, statt still zu `any` zu werden. **Direkt relevant:** genau das hätte uns gestoppt, wenn wir in `formatDate` vergessen hätten, `ts` zu typisieren — Demo 5s eigene "kein `any`"-Vorgabe ist im Grunde `noImplicitAny` in Textform.
- (auch dabei, bei uns aber praktisch wirkungslos, weil wir keine Klassen benutzen: `strictPropertyInitialization`, `strictBindCallApply`, `noImplicitThis`, `alwaysStrict`.)

An gelassen, weil ein Projekt, das gerade erst mit TypeScript anfängt, von Anfang an die strengste sinnvolle Basis haben sollte — nachträglich `strict` einzuschalten, wenn schon viel lockerer Code existiert, ist deutlich schmerzhafter als von Anfang an strikt zu sein und Fälle wie unseren `never[]`-Fund sofort zu sehen, statt sie Monate später zu entdecken.

F2: Unterschied zwischen einem Compile-Zeit-Typfehler und den Laufzeit-Bugs, die du in Übung 1 gefixt hast? Hätte TypeScript allein einen dieser konkreten Bugs fangen können? Warum (nicht)?

Einfach gesagt: ein Typfehler heißt "die FORM eines Werts widerspricht dem, was irgendwo erwartet wird" — komplett unabhängig davon, ob dein Programm zur Laufzeit das *Richtige* tut. Code kann 100 % typkorrekt sein und trotzdem jeden Bug aus UE1 enthalten.

Konkret durchgegangen:

- **Demo 2 (Referenz-Bug, `filteredEvidence = allEvidence`):** **Nein**, TypeScript hätte das nicht gefangen. Beide Variablen haben denselben, korrekten Typ (ein Array von Evidence-Objekten) — eine Referenz einer anderen Variablen zuzuweisen ist völlig gültiges, typkorrektes

JavaScript.

Der Bug ist eine Frage der **Objekt-Identität zur Laufzeit** (zwei Namen zeigen auf dasselbe Array), nicht der Form der Daten — dafür hat TypeScript's Typsystem kein Konzept.

- **Demo 3 (async-Bug, `evidenceViewLoading` nie auf `false` gesetzt): Nein.** Das ist eine fehlende Zeile, keine Typverletzung. `evidenceViewLoading: boolean` ist ein gültiger Typ, ob der

Wert *irgendwann* auf `false` gesetzt wird, ist eine Frage des **Kontrollflusses zur Laufzeit**, die TypeScript nicht verfolgt.

- **Demo 4 (stiller Bug, Promise geloggt statt `await`-item Wert): Auch hier nein** — in der Form, wie der Bug tatsächlich auftrat (`console.log(..., loadNoteAsync(...))`).

`console.log`

akzeptiert *jeden* Typ anstandslos, ein `Promise<string>` ist ein zulässiger Typ, kein Fehler. TypeScript *hätte* geholfen, wenn der Rückgabewert danach als `string` weiterverwendet worden

wäre (`firstNote.toUpperCase()` z. B. hätte TS abgelehnt) — aber genau das ist hier nicht passiert, der Wert wurde nur geloggt.

- **Demo 5.5 (kaputtes `localStorage` legt die App lahm, `JSON.parse` ohne `try/catch`):** Ebenfalls **nein**, und das aus einem besonders lehrreichen Grund: `JSON.parse` ist in TypeScript's eigener Standardbibliothek mit Rückgabotyp `any` deklariert — TypeScript kann beim Parsen von Text, dessen Inhalt es unmöglich vorher kennen kann, grundsätzlich nichts über

die Form des Ergebnisses aussagen. Dass der String überhaupt gültiges JSON ist, ist eine Laufzeit-Frage (`try/catch`), keine, die ein Typsystem beantworten kann.

Fazit: keiner der konkreten UE1-Bugs war ein Typfehler — sie waren Aliasing-, Kontrollfluss-, Timing- und Eingabe-Validierungs-Probleme. TypeScript ist eine **andere Achse** der Fehlerprävention als das, was UE1s Bug-Hunting-Demos gebraucht haben; es ersetzt Testen/Debuggen nicht, es ergänzt es um eine Kategorie von Fehlern, die diese vier Bugs zufällig alle nicht waren.

F3: Was macht `any` mit TypeScript's Prüfung für einen Wert, und warum hast du es in diesem ersten Durchgang vermieden, obwohl es schneller gewesen wäre, die Fehler einfach damit zum Schweigen zu bringen?

Einfach gesagt: `any` heißt "prüf hier gar nichts mehr" — sieht aus wie TypeScript, ist aber an dieser Stelle (und überall, wohin der Wert von da an fließt) einfach wieder JavaScript ohne Netz.

`any` schaltet für einen Wert **jede** Typprüfung ab: er darf jedem beliebigen Typ zugewiesen werden, jede Eigenschaft/Methode darf ungeprüft auf ihm aufgerufen werden, und diese "Prüfungslosigkeit" **wandert mit** — verbindet man einen `any`-Wert mit einem sauber getypten Wert, wird an dieser Stelle oft auch die Prüfung für den sauberen Wert stillschweigend ausgehebelt

("any ist ansteckend"). Man schreibt zwar `.ts`, aber genau dieser eine Wert ist wieder komplett ungeprüftes JavaScript.

Warum hier vermieden, obwohl schneller: hätten wir `stateJs as any` statt des engen `stateJs`

`as AppState` geschrieben, wären alle sieben echten Fehler aus Task 2 **sofort verschwunden** — nicht behoben, nur **unsichtbar gemacht**. Und ab dann hätte TypeScript uns bei jedem künftigen echten Fehler an genau dieser Stelle (z. B. `evidence.id` versehentlich mit einer Zahl statt einem Text verglichen, ein Feldname vertippt) ebenfalls nichts mehr gesagt — falsche Sicherheit ("hat ja kompiliert!"), ohne einen einzigen der eigentlichen Vorteile von TypeScript. Der enge Type-Assertion

(`as AppState`) hat genauso viel "Vertrauensvorschuss" gebraucht, prüft aber weiterhin **strukturell**

mit — ein Tippfehler im Feldnamen oder eine falsche Form wäre dort immer noch aufgefallen.

Demo 6 — Die Domain-Daten typisieren

(nicht präsentiert/angekreuzt, siehe Absprache — kurz gehalten)

`js/types.ts` mit den Domain-Interfaces angelegt, `js/data.ts` (Daten-Lader) konvertiert, das ambige Feld `Evidence.personIds` dokumentiert. Details in `UE2_CHANGES.md`.

F1: Das ambige Feld durchgehen — wie ist JavaScript ausgekommen, ohne sich auf eine Form festzulegen, und worauf hat TypeScript dich festgelegt?

`Evidence.personIds` ist fast überall eine echte `Person.id` (`"kernel-colt"`), aber E04 in `evidence.json` hat stattdessen den Anzeigenamen `"Nova Byte"` drinstehen. JavaScript kam damit

klar, weil `personIds` dort einfach ein Array war — Arrays aus Strings unterscheiden nicht zwischen "das ist eine ID" und "das ist ein Anzeigename", beides ist einfach ein String. Die bestehende `evidenceMentionsPerson()`-Funktion prüft deshalb pragmatisch beides.

TypeScript hat diese Inkonsistenz **nicht automatisch entdeckt** — `personIds: string[]` ist für eine ID genauso ein gültiger Typ wie für einen Namen, der Compiler kann den Unterschied nicht sehen. Worauf mich der **Versuch, den Typ ehrlich zu schreiben** aber festgelegt hat: ich musste mir die echten Daten nochmal anschauen und mich aktiv entscheiden, wie ich das dokumentiere — und

das Ergebnis (die Inkonsistenz + die Begründung, warum `evidenceMentionsPerson` beides prüft) steht

jetzt direkt als Kommentar am Feld in `types.ts`, statt nur implizit in einer Workaround-Funktion zu leben. Der Zwang kam also nicht vom Compiler selbst, sondern von der **Disziplin des Typ-Schreibens** — man kann bei einem benannten Interface-Feld nicht mehr "irgendwas reinlegen",

ohne sich wenigstens einmal zu fragen, was da wirklich reingehört.

F2: Gibt es ein Daten-Form-Problem, das TypeScript's statische Typen allein NICHT fangen können, weil die schlechten Daten erst zur Laufzeit aus einer JSON-Datei auftauchen? Was bräuchtest du zusätzlich?

Ja, ein sehr konkretes: `Evidence.status / Evidence.relevance` sind als Union-Typen typisiert (`"unreviewed" | "reviewed" | "flagged"` bzw. `"unknown" | "relevant" | "irrelevant"`) — sieht so aus, als würde das ungültige Werte verhindern. Die echten Daten in `evidence.json` widersprechen dem aber: `E12` hat tatsächlich `"Reviewed" / "Unknown"` (großgeschrieben) stehen, was **nicht** zu den erlaubten (kleingeschriebenen) Werten passt.

Warum TypeScript das trotzdem nicht meldet: wir laden die Daten über `(await res.json()) as Evidence[]` — `res.json()` liefert `any` (TypeScript's eigene Bibliotheksdefinition, siehe Demo 5 F2), und `as Evidence[]` ist eine **Zusicherung**, kein Test. TypeScript **vertraut** der Zusicherung, es öffnet die tatsächliche JSON-Datei nie und vergleicht nie, ob `"Reviewed"` wirklich zu `"reviewed" | "flagged" | "unreviewed"` passt. Ein Typfehler ist per Definition etwas, das der Compiler **im Code selbst** sieht — nicht etwas in einer Datendatei, die erst zur Laufzeit vom Server kommt. (Dass daraus in der App kein sichtbarer Bug wurde, liegt an `getStatusBadgeClass`, das den Wert vor dem Vergleich klein schreibt — reiner Zufall der bestehenden defensiven Programmierung, keine Absicherung durch TypeScript.)

Was zusätzlich nötig wäre: eine **Laufzeit-Validierung** — etwas, das die tatsächlich geladenen Daten *nach* dem `fetch()` wirklich inspiziert und mit der erwarteten Form vergleicht, z. B. eine Validierungs-Bibliothek (Zod, io-ts, ...) oder eine selbst geschriebene Prüf-Funktion, die bei einem unerwarteten Wert loggt/wirft. Typen allein schützen nur den Code, der die Daten **benutzt** (unter der Annahme, sie seien schon richtig geformt) — sie prüfen nie nach, ob die Bytes, die tatsächlich übers Netzwerk kommen, dieser Annahme entsprechen.

F3: Unterschied zwischen `interface` und `type` für eine Objekt-Form? Welches hast du für die Domain-Modelle benutzt, und spielt das hier überhaupt eine Rolle?

Unterschied: beide können eine Objekt-Form beschreiben, und für reine Objekt-Shapes sind sie praktisch austauschbar. Unterschiede, die es trotzdem gibt: ein `interface` kann später erneut geöffnet und um weitere Felder ergänzt werden ("declaration merging") und nutzt `extends`, um von

einem anderen zu erben; ein `type`-Alias kann das nicht (einmal definiert, fest), kann dafür aber **mehr als nur Objekt-Formen** ausdrücken — Vereinigungen (`"a" | "b"`), Tupel, primitive Aliase.

Was wir benutzt haben: `interface` für die fünf Domain-Entitäten (`Evidence`, `Person`, `Location`, `TimelineEvent`, `CaseFile`) — das sind "Dinge mit eigener Identität", der klassische Interface-Anwendungsfall, und konsistent zu dem, was wir in Demo 5 schon angefangen hatten. `type` dagegen für `ReviewStatus`, `Relevance`, `Certainty` — das sind **Vereinigungstypen** aus konkreten Text-Werten (`"unreviewed" | "reviewed" | "flagged"`).

Spielt das hier eine Rolle? Bei den fünf Domain-Entitäten: nein, reine Geschmackssache, `type` hätte genauso funktioniert. Bei `ReviewStatus / Relevance / Certainty`: **ja, zwingend** — ein `interface` kann grundsätzlich **keine** Vereinigung aus String-Literalen ausdrücken (ein Interface beschreibt immer eine Objekt-Form mit benannten Feldern, keinen "entweder-oder"-Wert). Für diese drei Fälle war `type` also nicht Stil, sondern die einzig mögliche Wahl.

Demo 7 — Die komplette App migrieren

(nicht präsentiert/angekreuzt, siehe Absprache — kurz gehalten)

Alle 10 verbliebenen `.js`-Module zu `.ts` konvertiert, App kompiliert erstmals als Ganzes mit 0 Fehlern, kein `any` irgendwo. Details in `UE2_CHANGES.md`.

F1: Zeig eine konkrete Stelle aus der Migration, an der TypeScript dich zu echtem Nachdenken gezwungen hat — nicht nur zu mechanischem Tippen von Typen.

Einfach gesagt: an den meisten Stellen war Konvertieren reine Fleißarbeit — Typ dazuschreiben, fertig. An genau einer Stelle (`window.navigateTo = ...` in `main.ts`) hat TypeScript aber einen echten Fehler gemeldet, für den es keine mechanische Lösung gab, sondern eine bewusste Entscheidung brauchte.

Konkrete Stelle: `main.ts` hängt sechs Funktionen an `window` (`window.navigateTo = navigateTo;` usw.), weil `index.html` noch inline `onClick="navigateTo(...)"`-Attribute aus der Zeit vor dem ES-Modul-Split (UE1) hat — diese Attribute können nur globale, auf `window` liegende Funktionen aufrufen. TypeScript's eingebautes `window`-Interface kennt unsere sechs Custom-Properties aber naturgemäß nicht: `window.navigateTo = ...` ist ohne Weiteres ein Fehler (`Property 'navigateTo' does not exist on type 'window'`).

Die **mechanische, aber falsche** Lösung wäre `(window as any).navigateTo = ...` gewesen — hätte den Fehler zum Schweigen gebracht, ohne irgendwas über die eigentliche Situation auszusagen. Stattdessen musste ich nachdenken: *warum* hängen diese sechs Funktionen überhaupt an `window`

(Antwort: die alten inline-`onClick`-Attribute, ein bewusst noch nicht bereinigter Rest aus einer früheren Demo), und *was ist der ehrliche Weg*, TypeScript das mitzuteilen? Die Antwort war TypeScript's eigener Mechanismus für genau diesen Fall — **Declaration Merging**: ein bereits bestehendes `interface` (hier `window`) erneut öffnen und um die eigenen Felder erweitern:

```
declare global {
  interface Window {
    navigateTo: (viewName: string) => void;
    // ... 5 weitere
  }
}
```

Der Unterschied zu `as any`: jede der sechs Zuweisungen bleibt danach weiterhin **strukturell geprüft** — eine falsche Parameteranzahl oder ein falscher Rückgabetyt würde TypeScript immer noch melden. Nur die *Existenz* dieser sechs zusätzlichen Felder auf `window` wird zugesichert, nicht jede Prüfung an dieser Stelle abgeschaltet.

F2: Wann ist `any` tatsächlich die richtige Wahl, und wann ist es ein Zeichen, dass man den Typ eigentlich richtig modellieren sollte? Wo genau hast du in dieser Migration die Grenze gezogen?

Einfach gesagt: `any` wäre an jeder Stelle dieser Migration die *schnellere* Lösung gewesen — aber "schnell einen roten Fehler wegmachen" und "den Typ tatsächlich richtig verstehen" sind zwei

verschiedene Dinge. Die Grenze, die ich gezogen habe: sobald ich **wusste**, welche Form ein Wert wirklich hat (auch wenn TypeScript das selbst nicht herleiten konnte), war eine gezielte Zusicherung (`as x`) der richtige Weg — `any` wäre dort nur Bequemlichkeit auf Kosten der Sicherheit gewesen.

Grundsätzlich gilt: `any` ist gerechtfertigt, wenn ein Wert **wirklich** zur Compile-Zeit jede beliebige Form haben könnte und es keine sinnvolle engere Aussage gibt, die man machen kann — z. B.

das direkte, ungeprüfte Ergebnis eines `JSON.parse()` auf beliebigem Fremd-Input, bevor man es überhaupt angeschaut hat. Es ist dagegen ein **Warnsignal**, dass man den Typ eigentlich richtig modellieren sollte, wenn man in Wahrheit sehr wohl weiß, was für eine Form der Wert hat — man ihn

TypeScript nur (noch) nicht auf eine Art mitgeteilt hat, die der Compiler selbst herleiten kann.

Wo genau ich die Grenze gezogen habe, an den zwei konkretesten Fällen:

- `(e.target as HTMLSelectElement).value as ReviewStatus` (`views/evidence.ts`): kein `any`, weil ich zwei Dinge sicher wusste — erstens, dass das Change-Event von genau diesem `<select>` kommt (nicht irgendeinem beliebigen Element), zweitens, dass die drei `<option value="...">` dieses `<select>`s von **unserem eigenen Code** erzeugt werden (`statusOptionHTML`, ein paar Zeilen weiter oben in derselben Datei) — der Wertebereich ist also tatsächlich exakt `ReviewStatus`, nur über zwei getrennte Funktionen verteilt, die TypeScript nicht miteinander verknüpfen kann. Eine gezielte Zusicherung ist hier ehrlich, `any` wäre eine Notlüge gewesen.
- `JSON.parse(raw)` **beim Laden von** `localStorage` (`storage.ts`, `workspace.ts`): hier **habe** ich bewusst einen Zwischenschritt über `unknown` gemacht statt direkt zu casten (`const parsed: unknown = JSON.parse(raw); ... Array.isArray(parsed) ? (parsed as string[]) :` `[]`), weil ich hier *nicht* sicher weiß, was wirklich drinsteht — der Nutzer könnte den `localStorage`-Eintrag von Hand editiert oder eine ältere App-Version könnte ein anderes Format

geschrieben haben. `unknown` zwingt dazu, den Wert erst zu **prüfen** (`Array.isArray`, `typeof === "object"`), bevor man ihn benutzt — der sicherere Zwischenschritt zwischen "ich weiß es genau" (direkte Zusicherung) und "ich weiß es gar nicht" (`any`).

Fazit der Grenze: `as x` überall dort, wo eine Zusicherung durch echtes Wissen über die Quelle des Werts gedeckt ist (unser eigener Code erzeugt die Daten). `unknown` + expliziter Laufzeit-Check dort, wo die Quelle außerhalb unserer Kontrolle liegt (Nutzereingabe, `localStorage`, Netzwerk-Antwort). `any` an **keiner** Stelle — es gab in der ganzen Migration keinen Fall, an dem ich wirklich nichts über die erwartete Form eines Werts wusste.

F3: Hat die Migration einen echten, vorher unbemerkten Bug aufgedeckt — oder war es reines "Rauschen" (Fehler, die nur TypeScript's Pedanterie waren, aber nie ein reales Problem gewesen wären)?

Einfach gesagt: überwiegend Rauschen — TypeScript, das auf Dinge besteht, die zur Laufzeit nie wirklich ein Problem gewesen wären (z. B. `getElementById("view-" + hash)!`, wo der Code selbst

schon durch die `validViews`-Prüfung sicherstellt, dass das Element existiert). Aber **ein** Fund war genau die Art Stelle, die TypeScript-Migrationen eigentlich rechtfertigt: der

`window.navigateTo = ...`-Fall aus F1 war kein *Laufzeit*-Bug (die App funktionierte vorher einwandfrei), aber er hat eine **stillschweigende, nirgends dokumentierte Abhängigkeit** sichtbar

gemacht, die vorher komplett implizit war — dass `index.html`'s inline-`onClick`-Attribute sich darauf verlassen, dass `main.js` beim Laden genau diese sechs Namen an `window` hängt. In reinem

JavaScript hätte ein Tippfehler in einem dieser sechs Namen (z. B. `window.navigateto` statt

`window.navigateTo`) erst beim **Klicken** im Browser als `"navigateTo is not a function"`

aufgeschlagen — jetzt steht die vollständige Liste explizit als Typ in `main.ts`, und ein

Tippfehler wäre sofort beim `npm run typecheck` aufgefallen, lange bevor irgendjemand geklickt hätte.

Die meisten übrigen Funde waren dagegen tatsächlich reines Rauschen im Sinne der Frage — echte

Compiler-Meldungen, aber ohne realen Bug dahinter:

- `document.getElementById("view-" + hash)!` in `navigation.ts`: die `!`-Zusicherung ist reine "Compiler-Pedanterie" (mein eigener Kommentar im Code) — der Code stellt über die `validViews`-Liste ohnehin schon sicher, dass das Element existiert, TypeScript kann das nur nicht über die String-Verkettung hinweg selbst sehen.
- Die ganzen `noUncheckedIndexedAccess`-Meldungen bei `nodeList`-Index-Zugriffen: in jedem einzelnen Fall war der Index tatsächlich immer gültig (die Schleife lief ja genau `list.length` mal) — TypeScript kann das nur grundsätzlich nicht beweisen, unabhängig vom konkreten Code.

Fazit: die Migration war in erster Linie eine **Absicherung nach vorne** (künftige Tippfehler an genau diesen Stellen werden jetzt sofort gemeldet), nicht primär eine Bug-Jagd — anders als z. B. die `noUncheckedIndexedAccess`-Funde in Demo 5/6 gab es in Demo 7 keinen Fall, wo ein tatsächlich fehlerhaftes Laufzeitverhalten aufgedeckt wurde. Der `window`-Fund ist trotzdem mehr als reines Rauschen: er macht eine vorher unsichtbare Kopplung zwischen zwei Dateien (`index.html` und `main.ts`) zum ersten Mal explizit und maschinenprüfbar.

Demo 8 — GitHub Actions: Development-Workflow

`.github/workflows/ci.yml` neu angelegt (Trigger, Checkout, Node-Setup mit Caching, `lint` + `format:check`), `package.json` um das Script `format:check` ergänzt. Details, die komplette Datei tabellarisch erklärt und der Fail→Fix-Ablauf in `UE2_CHANGES.md`.

F1: Was ist der Unterschied zwischen einem Workflow, einem Job und einem Step in GitHub Actions? Zeig auf je eins in deiner Workflow-Datei.

Einfach gesagt: Workflow = das ganze Rezeptbuch-Kapitel ("wann backe ich überhaupt"), Job = ein einzelnes Rezept darin (bekommt eine eigene, saubere Küche), Step = eine einzelne Anweisung im Rezept ("Ofen vorheizen"). Die Steps eines Jobs laufen der Reihe nach in derselben Küche.

Ebene	Definition	Stelle in <code>.github/workflows/ci.yml</code>
Workflow	Die komplette YAML-Datei — legt fest, <i>wann</i> überhaupt etwas laufen soll (<code>on:</code>) und was danach passiert	die ganze Datei, benannt über <code>name: CI</code>
Job	Eine Gruppe von Steps, die zusammen eine eigene, frische virtuelle Maschine bekommt (<code>runs-on:</code>). Mehrere Jobs im selben Workflow laufen standardmäßig parallel , jeder auf seiner eigenen Maschine	<code>lint-and-format:</code> — bei uns nur ein einziger Job
Step	Ein einzelner Befehl innerhalb eines Jobs. Steps laufen nacheinander , auf derselben Maschine, und teilen sich deren Dateisystem-Zustand (was Step 1 auf die Platte schreibt, sieht Step 2 noch)	z. B. <code>- name: Node.js einrichten / uses: actions/setup-node@v4</code> — einer von fünf Steps im Job

Konkret bei uns: **ein** Workflow, **ein** Job, **fünf** Steps (Auschecken → Node einrichten → Installieren → Linter laufen lassen → Formatierung prüfen). Der Job-Level ist der Grund, warum z. B. `npm ci` (Step 3) die Pakete installiert, die `npm run lint` (Step 4) direkt danach benutzen kann — beide Steps laufen im selben Job, auf derselben Maschine, mit demselben `node_modules/`.

Ein **zweiter** Job (hätten wir z. B. Linting und Formatierung in getrennte Jobs aufgeteilt) hätte dagegen bei null angefangen und `npm ci` erneut gebraucht — Jobs teilen sich nichts automatisch.

F2: Warum sollte lint/format überhaupt in CI laufen, wenn es ohnehin (oder zumindest potenziell) auf der eigenen Maschine jedes Entwicklers vor dem Push läuft?

Einfach gesagt: "könnte lokal laufen" ist nicht dasselbe wie "läuft garantiert lokal". CI ist die eine Stelle, die **niemand** versehentlich überspringen kann — weil sie nicht auf irgendjemandes Rechner läuft, sondern zentral bei GitHub, bei jedem einzelnen Push, auf einer immer gleichen, sauberen Maschine.

Konkrete Gründe, warum "läuft ja eh lokal" allein nicht reicht:

- **Menschen vergessen Dinge.** Zeitdruck, ein schneller Hotfix, ein `git push` mitten in einer anderen Sache — `npm run lint` lokal auszuführen ist ein **freiwilliger** Schritt, den man überspringen kann, ohne dass irgendetwas es verhindert. Ein CI-Schritt lässt sich dagegen nicht "vergessen" — er läuft, ob man daran gedacht hat oder nicht.
- **"Lokal" ist nicht garantiert dieselbe Umgebung.** Ohne CI verlässt man sich darauf, dass jede Person, die pusht, dieselbe Node-Version, dieselben installierten Paketversionen und eine funktionierende lokale Tool-Konfiguration hat. CI installiert bei **jedem** Lauf über `npm ci` exakt das aus der committeten Lockfile (Demo 1) — ein objektiver, für alle identischer Maßstab, unabhängig davon, was gerade zufällig auf wessen Rechner installiert ist.
- **Ein Pull Request von jemand anderem hat vielleicht gar keine lokale Umgebung, die du prüfen kannst.** Sobald mehr als eine Person am Repo arbeitet (oder ein externer Beitrag per PR kommt), kannst du nicht kontrollieren, ob die andere Person `lint / format` vor dem Push überhaupt ausgeführt hat. Der `pull_request`-Trigger sorgt dafür, dass **jeder** Beitrag denselben automatischen Check durchläuft, bevor er gemergt werden darf — unabhängig vom Workflow der einreichenden Person.
- **Ein objektives, dauerhaftes, für alle sichtbares Protokoll.** Der Actions-Tab zeigt für **jeden** Commit einen grünen Haken oder ein rotes Kreuz — eine nachvollziehbare, permanente Historie, die ein rein lokaler Lauf nie hinterlässt (der ist nach dem Terminal-Fenster schließen weg).

Fazit: lokal laufen lassen ist die **schnelle Rückmeldung** beim Programmieren selbst (Editor, manueller `npm run lint`), CI ist das **erzwungene, nicht überspringbare, für alle gleiche** Gate, das am Ende wirklich garantiert, dass nichts Kaputtes durchrutscht — die beiden ersetzen sich nicht gegenseitig, sie ergänzen sich.

F3: Was macht Dependency-Caching in deinem Workflow, und was würde (sowohl bei der Korrektheit als auch bei der Geschwindigkeit) passieren, wenn du es entfernen würdest?

Einfach gesagt: jeder CI-Lauf startet auf einer komplett leeren, frisch hochgefahrenen Maschine — nichts vom letzten Mal ist mehr da. Caching hebt genau den einen Ordner auf, in dem npm heruntergeladene Pakete zwischenspeichert, damit der nächste Lauf sie nicht erneut aus dem Internet laden muss.

Was `cache: "npm"` (Teil von `actions/setup-node@v4`) **konkret tut:** `npm ci` lädt jedes Paket aus `package-lock.json` aus dem npm-Registry herunter und legt es zusätzlich in einem lokalen Download-Cache-Ordner (`~/.npm`) ab, bevor es nach `node_modules/` entpackt/verlinkt wird. Auf einer

Wegwerf-CI-Maschine wäre dieser `~/.npm`-Ordner nach jedem Lauf sofort wieder weg. `cache: "npm"`

lässt GitHub genau diesen einen Ordner nach dem Lauf als "Actions-Cache" sichern und beim

nächsten Lauf automatisch wieder einspielen — **solange sich der Cache-Schlüssel nicht geändert hat**. Der Cache-Schlüssel wird automatisch aus einem Hash von `package-lock.json` berechnet: ändert sich auch nur eine Version in der Lockfile, bekommt der nächste Lauf automatisch

einen **neuen** Cache-Eintrag (leer, muss einmal neu laden) statt versehentlich einen veralteten zu benutzen.

Ohne Caching:

- **Korrektheit: unverändert.** `npm ci` installiert so oder so **exakt** das, was in `package-lock.json` steht — ob die Pakete aus einem Cache oder frisch aus dem Registry kommen, ändert am Ergebnis (welche Versionen landen in `node_modules/`) nichts. Caching ist reine Performance-Optimierung, keine Korrektheits-Frage.
- **Geschwindigkeit: spürbar langsamer**, weil **jeder einzelne** Push wieder **jedes** Paket komplett neu aus dem npm-Registry über das Internet herunterladen müsste, statt die meisten davon aus einem lokal (auf dem Runner) bereits vorhandenen Cache zu nehmen. Bei unserem kleinen Abhängigkeitsbaum (aktuell ~7 Pakete, siehe `package.json`) ist der Unterschied gering, aber bei einem realen Projekt mit hunderten transitiven Abhängigkeiten (Demo 1, F4) macht das ohne Cache oft den Unterschied zwischen einem 10-Sekunden- und einem Minuten-langen Install-Schritt — bei jedem einzelnen Workflow-Lauf, jeden Tag.

Fazit: Caching ist hier ausschließlich ein **Geschwindigkeits**-Hebel, kein

Korrektheits-Mechanismus — genau deshalb ist es unbedenklich, es einfach als optionales `cache: "npm"` dazuzuschreiben, ohne Angst, dass es je ein falsches, veraltetes Ergebnis verschleiern.

Demo 9 — GitHub Actions: Deployment-Workflow

`.github/workflows/deploy.yml` neu angelegt (zwei Jobs: `build` → `deploy`, Deploy über `actions/upload-pages-artifact` + `actions/deploy-pages`), `vite.config.js` um `base: "/Mystery-Road-AWE-2026-lukesch/"` ergänzt (sonst 404 für alle Assets unter dem GitHub-Pages-Unterpfad), Pages-Quelle in den Repo-Settings auf "GitHub Actions" umgestellt. Details, komplette Datei tabellarisch erklärt, in `UE2_CHANGES.md`.

F1: Warum führt der Deploy-Workflow `lint` und `build` selbst nochmal aus, statt "hat ja schon auf meinem Rechner geklappt" zu vertrauen oder direkt das Ergebnis von Demo 8s Workflow wiederzuverwenden?

Einfach gesagt: "hat bei mir funktioniert" ist keine Garantie, die ein automatischer Veröffentlichungs-Schritt akzeptieren darf — und Demo 8s Workflow-Lauf war eine **andere** Maschine zu einem **anderen** Zeitpunkt mit potenziell schon wieder anderem Code-Stand. Der Deploy-Workflow baut sich deshalb sein **eigenes, frisches, garantiert aktuelles** Ergebnis, statt irgendjemandem zu vertrauen.

Warum nicht "hat ja schon auf meinem Rechner geklappt" reicht: genau das ist der Grund, warum

CI überhaupt existiert (siehe Demo 8, F2) — ein manueller, lokaler Lauf ist nicht reproduzierbar garantiert, könnte übersprungen worden sein, oder auf einem Rechner mit leicht anderen installierten

Versionen gelaufen sein. Ein Deploy-Schritt, der das nicht selbst nochmal prüft, würde diese ganze Garantie wegwerfen, genau an der Stelle, wo sie am meisten zählt (etwas geht **live**, öffentlich sichtbar).

Warum nicht einfach Demo 8s Workflow-Ergebnis wiederverwenden:**** zwei konkrete Gründe:

- **Zeitliche Lücke.** Demo 8s `ci.yml` läuft bei jedem Push/PR — sein letzter erfolgreicher Lauf könnte von einem **älteren** Commit stammen als dem, der gerade auf `main` gelandet ist (z. B. wenn zwischen PR-Freigabe und tatsächlichem Merge noch etwas dazwischenkommt).
"Neuester grüner Lauf" ist nicht dasselbe Versprechen wie "genau der Code, der jetzt deployt wird, wurde gerade geprüft".
- `ci.yml` **erzeugt gar kein wiederverwendbares Ergebnis.** Es prüft nur (`lint`, `format:check`) und erzeugt **keinen** `dist/`-Build — es gibt technisch nichts, was `deploy.yml` von dort "übernehmen" könnte. Selbst wenn man das ändern wollte (z. B. Build-Artefakte zwischen Workflows teilen), bräuhete man einen expliziten, riskanteren Mechanismus (Artefakte hochladen/herunterladen über Workflow-Grenzen hinweg) statt der einfachen, robusten Lösung: **beides einfach nochmal selbst tun**, in derselben, garantiert aktuellen Umgebung.

Fazit: Redundanz ist hier **bewusst** — der Deploy-Workflow verlässt sich auf **nichts**, was vorher irgendwo anders gelaufen ist, sondern stellt sein eigenes, minimal notwendiges "ist der aktuelle Code wirklich in Ordnung?"-Ergebnis her, direkt bevor er live geht.

F2: Was ist der tatsächliche Mechanismus, über den dein Deploy-Workflow auf GitHub Pages veröffentlicht (z. B. eine dedizierte Deploy-Action, die ein Artefakt veröffentlicht, ein Push auf einen `gh-pages`-Branch, oder etwas anderes)? Konkret erklären, was das tut.

Einfach gesagt: kein `git push` auf einen extra Branch (das wäre der ältere, verbreitete Weg) — stattdessen ein **Artefakt-Upload**: `build` verpackt `dist/` in ein spezielles Paket und lädt es zu GitHub hoch, `deploy` nimmt genau dieses Paket und schaltet es live. Zwei getrennte Schritte, kein Umweg über Git selbst.

Konkret, Schritt für Schritt:

1. `actions/configure-pages@v5` (im `build`-Job) fragt bei GitHub die aktuelle Pages-Konfiguration des Repos ab (u. a. die Basis-URL) — reine Vorbereitung, verändert noch nichts.
2. `actions/upload-pages-artifact@v3` (im `build`-Job, `path: dist`) packt den Inhalt von `dist/` in ein **GitHub-Actions-Artefakt** — ein von GitHub selbst verwaltetes, temporäres Speicherobjekt, technisch ähnlich den "Build-Artefakten", die man sich sonst auch manuell von einem Workflow-Lauf herunterladen kann. An dieser Stelle ist **noch nichts live** — das Artefakt liegt nur bereit.
3. `needs: build` lässt den `deploy`-Job erst starten, wenn dieses Artefakt fertig hochgeladen und `build` insgesamt erfolgreich war.
4. `actions/deploy-pages@v4` (im `deploy`-Job) ist die eigentliche Veröffentlichung: die Action holt sich das zuvor hochgeladene Artefakt und übergibt es an GitHubs **Pages-API**, die es live unter der Pages-URL schaltet. Die Authentifizierung dabei läuft über **OIDC** (`permissions: id-token: write`) — der Workflow-Lauf bekommt von GitHub selbst einen kurzlebigen, automatisch ausgestellten Token, mit dem er sich gegenüber der Pages-API als "ich bin genau dieser eine, gerade laufende Workflow-Lauf dieses Repos" ausweist. Kein von Hand angelegtes, dauerhaftes Secret nötig (Unterschied dazu: Demo 10, F2).

Warum nicht der ältere `gh-pages`-Branch-Weg (ein Tool committet den Build-Output als Dateien

auf einen separaten Git-Branch namens `gh-pages`, GitHub Pages liefert dann diesen Branch aus)? Beide Wege funktionieren, aber der Artefakt-Weg (den wir benutzen) ist der von GitHub selbst aktuell

empfohlene, offizielle Standard-Mechanismus (`actions/deploy-pages`, seit 2022 verfügbar): kein zusätzlicher, künstlicher Git-Branch voller generierter Build-Dateien, der die eigentliche Commit-Historie unübersichtlich macht, keine Notwendigkeit, dass der Workflow selbst Schreibrechte

auf den Git-Inhalt des Repos braucht (nur auf die Pages-**Veröffentlichung** über die API, `pages: write` statt `contents: write`) — strikter abgegrenzte, genauer zugeschnittene Rechte.

F3: Was müsstest du ändern, wenn du stattdessen auf einen anderen statischen Host deployen würdest (z. B. Netlify, Vercel, ein einfacher Server per SFTP)? Was würde gleich bleiben?

Einfach gesagt: der **Bau-Teil** (Checkout, Node einrichten, `npm ci`, `npm run lint`, `npm run build`) bleibt zu 100 % gleich, egal wohin am Ende deployt wird — der bekommt von der Zielplattform gar nichts mit, er erzeugt einfach `dist/`. Ändern müsste sich ausschließlich der **letzte Teil**: wie genau `dist/` an sein Ziel kommt.

Bleibt gleich	Ändert sich
<code>actions/checkout@v4</code> , <code>actions/setup-node@v4</code> , <code>npm ci</code> , <code>npm run lint</code> , <code>npm run build</code> — der komplette <code>build</code> -Job bis zum fertigen <code>dist/</code> -Ordner	der <code>deploy</code> -Job komplett: andere Action(s), andere Authentifizierung, andere <code>permissions</code> :
<code>vite.config.js</code> <code>base</code> -Grundidee (Vite muss wissen, unter welchem Unterpfad die App live läuft)	der konkrete Wert von <code>base</code> — Netlify/Vercel liefern typischerweise unter der Wurzel (<code>/</code>) einer eigenen Domain aus, <code>base: "/"</code> (der Vite- Standard) würde dort reichen, kein <code><repo-name>/</code> -Unterpfad nötig
die Grund-Idee "bauen, dann veröffentlichen" (zwei Phasen)	der genaue Mechanismus der Veröffentlichung selbst

Konkret bei den drei genannten Alternativen:

- **Netlify/Vercel:** beide bieten eigene, offizielle GitHub-Actions (`netlify/actions-cli` bzw. `amondnet/vercel-action`, oder alternativ deren eigene Git-Integration ganz ohne GitHub Actions).
Statt `pages: write` / `id-token: write` + `actions/deploy-pages` bräuchte man ein **API-Token** dieser Plattform, gespeichert als GitHub-**Secret** (`Settings` → `Secrets and variables` → `Actions`) und im Workflow über `secrets.NETLIFY_AUTH_TOKEN` / `secrets.VERCEL_TOKEN` referenziert —
ein grundlegend anderes Rechte-Modell als das OIDC-basierte GitHub-Pages-Setup (siehe Demo 10 F2
für den direkten Vergleich Secret vs. OIDC).
- **Ein einfacher Server per SFTP:** kein "Deploy-Provider" mit eigener Action, eher ein generischer SFTP-Upload-Schritt (z. B. `uses: SamKirkland/FTP-Deploy-Action`), der `dist/` einfach auf den Server hochlädt. Bräuchte Server-**Zugangsdaten** (Host, Benutzername, Passwort oder SSH-Key) —
ebenfalls als GitHub-Secrets hinterlegt, **nicht** im Workflow-Code selbst (siehe Demo 10 F2,

wo

genau diese Unterscheidung zwischen Secrets und den `permissions:`-Rechten Thema ist).

Fazit: der Ortswechsel des Hostings ist überraschend klein im Code — man tauscht im Wesentlichen

nur die letzten paar Zeilen (den `deploy`-Job) aus, weil "wie baue ich die App" und "wohin veröffentliche ich sie" von Anfang an bewusst zwei getrennte Jobs waren.

Demo 10 — Workflow-Trigger, Rechte & Fehlerfälle

Echten TS-Fehler (geparkte Demo-5-Testzeile in `js/utis.ts`) reaktiviert, gepusht, beobachtet:

`build`-Job wird rot, `deploy` läuft gar nicht erst an, die alte Live-Seite bleibt währenddessen online. Fehler wieder behoben, gepusht, beide Workflows wieder grün. Rechte/Secrets des Deploy-Workflows tabellarisch durchgegangen. Details, Screenshots-Beschreibung in `UE2_CHANGES.md`.

F1: Wenn dein Build-Schritt fehlschlägt — bleibt die zuvor deployte Version der App live, wird sie offline genommen, oder passiert etwas anderes? Ist das das gewünschte Verhalten, und warum?

Einfach gesagt: die alte Version bleibt einfach stehen — ein kaputter Bauversuch **überschreibt nichts**. Genau live beobachtet: während `build` in Demo 10 rot war, lief die App unter der GitHub-Pages-URL unverändert mit dem alten Stand weiter, kein Ausfall, keine leere Seite.

Warum das so ist (der Mechanismus): `deploy` hat `needs: build` — startet also **nur**, wenn `build` erfolgreich durchgelaufen ist. Schlägt `build` fehl, wird `actions/deploy-pages@v4` (der einzige Schritt, der die Live-Seite tatsächlich verändert) **gar nicht erst ausgeführt**. Die zuvor veröffentlichte Version bleibt exakt so bestehen, wie sie beim letzten **erfolgreichen** Lauf veröffentlicht wurde — GitHub Pages "vergisst" die alte Version nicht einfach, weil ein neuer Versuch angefangen (und dann abgebrochen) wurde.

Ist das das gewünschte Verhalten? Ja, eindeutig:

- **Nutzer:innen der Seite merken nichts von einem fehlgeschlagenen internen Build.**
Wäre es andersrum (Seite geht sofort offline, sobald irgendein Build-Versuch scheitert), würde ein einziger Tippfehler im Code die komplette öffentliche Seite lahmlegen, bis jemand den Fehler behebt — ein winziges internes Problem hätte ein unverhältnismäßig großes, sofort sichtbares Symptom nach außen.
- **"Alt, aber funktionierend" ist fast immer besser als "kaputt oder leer".** Die alte Version war zuletzt bewiesenermaßen ein Zustand, der den kompletten Build-Prozess (Lint, TypeScript, Vite Build) erfolgreich durchlaufen hat — sie ist also mit Sicherheit funktionsfähig. Eine neue, fehlerhafte Version wäre das Gegenteil.
- Das ist genau der Sinn von `needs:` zwischen zwei Jobs: **erst prüfen, dann erst verändern**

nie eine Veröffentlichung riskieren, deren Voraussetzung (ein erfolgreicher Build) nicht erfüllt ist.

F2: Welche GitHub-Actions-Rechte und/oder Secrets braucht dein Deploy-Workflow wirklich, und wo hast du sie vergeben/gespeichert? Was ist das Sicherheitsrisiko, wenn man hier zu großzügig Rechte vergibt?

Einfach gesagt: drei Zeilen Rechte, **null** Secrets. Alles steht direkt sichtbar im `permissions:`-Block von `deploy.yml` — kein verstecktes Passwort irgendwo in den Repo-Settings.

Was	Zweck	Wo konfiguriert
<code>permissions:</code> <code>contents:</code> <code>read</code>	Repo-Inhalt lesen (für <code>actions/checkout</code>)	<code>.github/workflows/deploy.yml</code>
<code>permissions:</code> <code>pages: write</code>	über die GitHub-Pages-API tatsächlich veröffentlichen dürfen	ebenda
<code>permissions:</code> <code>id-token:</code> <code>write</code>	sich per OIDC gegenüber der Pages-API als "ich bin dieser eine, gerade laufende Workflow-Lauf" ausweisen	ebenda
GitHub-Pages-Quelle: "GitHub Actions"	erlaubt Workflows überhaupt, auf Pages zu veröffentlichen (statt eines Branches)	Repo → Settings → Pages
Secrets	keine	<code>Settings → Secrets and variables → Actions</code> bleibt für dieses Setup leer

Warum keine Secrets nötig sind: GitHub Pages ist ein **GitHub-eigenes** Ziel. Statt eines dauerhaften, von Hand angelegten API-Tokens (das irgendwo sicher gespeichert, irgendwann erneuert, und bei einem Leak sofort widerrufen werden müsste) bekommt jeder einzelne Workflow-Lauf von GitHub selbst automatisch einen **kurzlebigen** OIDC-Token ausgestellt — gültig nur für die Dauer dieses einen Laufs, danach automatisch wertlos. `id-token: write` ist die Erlaubnis, diesen Token überhaupt anzufordern.

Sicherheitsrisiko bei zu großzügigen Rechten: GitHub Actions vererbt standardmäßig (wenn man gar keinen `permissions:`-Block schreibt) oft recht weitreichende Rechte an den automatisch erzeugten `GITHUB_TOKEN` — je nach Repo-Voreinstellung z. B. Schreibrechte auf Issues, Pull Requests, sogar den Repo-Inhalt selbst. Das eigentliche Risiko liegt nicht in diesem einen Workflow selbst (den habe ich ja kontrolliere), sondern darin, **was bei einer kompromittierten Abhängigkeit passieren könnte:** läuft in `npm ci / npm run build` irgendwo eine böartige (oder gehackte) npm-Paket-Version mit, könnte deren Code — falls der Workflow zu großzügige

Rechte

hat — diese Rechte missbrauchen (z. B. Code auf `main` pushen, Repo-Einstellungen ändern, in anderen Workflows lesen). Mit **eng zugeschnittenen** Rechten (`contents: read` statt `write`, kein `issues: / pull-requests:`-Zugriff, den dieser Workflow gar nicht braucht) ist der maximale Schaden im Ernstfall drastisch kleiner — genau das Prinzip "**least privilege**" (minimal nötige Rechte), das hinter dem expliziten, knappen `permissions:`-Block steht, statt sich auf GitHub-Voreinstellungen zu verlassen.

F3: Was ist der Unterschied zwischen `on: push`, `on: pull_request` und `on: workflow_dispatch`? Welchen hast du für den Development-Workflow (Demo 8) und welchen für den Deployment-Workflow (Demo 9) benutzt, und warum ist genau diese Zuordnung richtig?

Einfach gesagt: `push` = "irgendwer hat gerade Code irgendwohin geschoben", `pull_request` = "jemand will Code **zusammenführen**, prüf das vorher", `workflow_dispatch` = "ein Mensch klickt manuell einen Knopf". Drei verschiedene Auslöser für drei verschiedene Situationen.

Trigger	Läuft wann	Bei uns benutzt in
<code>on: push</code>	bei jedem tatsächlichen Push auf einen Branch (bei uns: jeden Branch, keine Einschränkung)	<code>ci.yml</code> (Demo 8) — und <code>deploy.yml</code> (Demo 9), dort aber eingeschränkt auf <code>branches: [main]</code>
<code>on: pull_request</code>	wenn ein Pull Request gegen den angegebenen Ziel-Branch geöffnet/aktualisiert wird — läuft gegen den gemergten Vorschau-Stand (PR-Branch + Ziel-Branch), nicht erst nach dem Merge	<code>ci.yml</code> (Demo 8), <code>branches: [main]</code> — prüft Beiträge, bevor sie gemergt werden dürfen
<code>on: workflow_dispatch</code>	nur, wenn ein Mensch im Actions-Tab explizit auf " Run workflow " klickt (oder per API)	<code>deploy.yml</code> (Demo 9) — zusätzlich zu <code>push</code> , als manueller Nachtrigger

Warum genau diese Zuordnung die richtige ist:

- **Demo 8 (`ci.yml`) auf `push` + `pull_request`:** der Dev-Workflow soll **so früh wie möglich** Feedback geben — auf **jedem** Branch, bei **jedem** Push, damit man Probleme sofort merkt, während man noch daran arbeitet, nicht erst wenn man versucht zu mergen. `pull_request` zusätzlich, weil das der Moment ist, an dem ein Beitrag **offiziell zur Aufnahme vorgeschlagen** wird — der letzte, verbindliche Check vor dem Merge, unabhängig davon, ob die einreichende Person vorher lokal geprüft hat.

- **Demo 9** (`deploy.yml`) **nur auf** `push` **gegen** `main`, **plus** `workflow_dispatch`: Deployment ist ein **Ergebnis**, kein Zwischenschritt — es soll **nicht** bei jedem Feature-Branch-Push passieren (dann würde ständig unfertiger, halb getesteter Code live gehen), und **nicht** schon bei einem offenen Pull Request (der ist ja noch gar nicht gemergt — würde man dort deployen, könnte jeder offene, noch nicht akzeptierte PR die Live-Seite überschreiben). Nur ein tatsächlicher Merge/Push auf `main` — der Branch, der laut Konvention immer der "fertige, freigegebene" Stand ist — soll live gehen. `workflow_dispatch` obendrauf ist rein praktisch: ein manueller Nachtrigger für Fälle, in denen man ohne neuen Commit nochmal deployen will (z. B. nach einer reinen Konfigurationsänderung in den Pages-Settings, oder für eine Live-Vorführung).

Warum wäre die vertauschte Zuordnung falsch? `deploy.yml` auf `pull_request` zu triggern wäre riskant (unauthorisierter/unreviewter Code aus einem PR würde live gehen, bevor überhaupt jemand zugestimmt hat) und würde zudem für **jeden** offenen PR parallel eine eigene Veröffentlichung versuchen — es gibt aber nur **eine** Live-Seite. `ci.yml` **nur** auf `workflow_dispatch` zu beschränken würde den ganzen Sinn von CI zunichtemachen: die Prüfung müsste dann jedes Mal von Hand angestoßen werden — genau das manuelle "könnte vergessen werden"-Problem, das CI laut Demo 8 (F2) eigentlich lösen soll.